

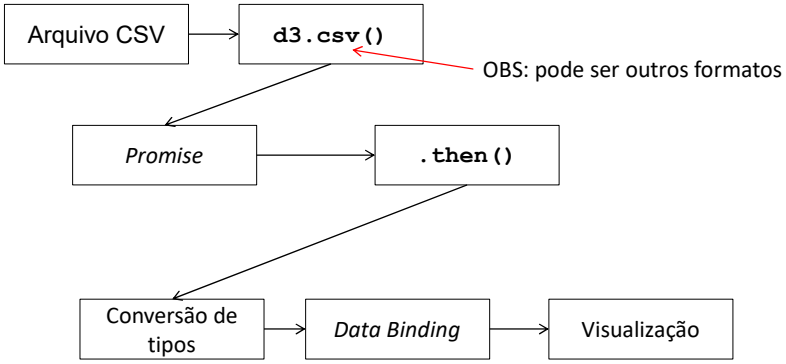
Associação de Dados Externos

José Gustavo de Souza Paiva

Leitura de Dados

- D3 oferece recursos para trabalhar com dados criados pelo código ou lidos de fontes externas
- Possibilidades
  - `d3.csv()`
  - `d3.json()`
  - `d3.tsv()`
  - `d3.xml()` (incluíndo gráficos SVG)
  - `d3.dsv()`
  - `d3.text()`

Fluxo de Dados Típico



Leitura de Dados - Observações

- Leitura dos dados é realizada de forma assíncrona
  - Ao solicitar a leitura do arquivo, será retornado uma ***Promise***
  - Esse objeto *Promise* será tratado, e o dado será retornado (caso não haja nenhum problema)
  - Navegador não pára processamento da página durante leitura do conteúdo do arquivo
- Navegadores mais modernos não permitem por padrão o acesso a dados locais
  - Uma maneira para executar os códigos é criando um servidor WEB local

Opção de Servidor WEB Local

- Servidor WEB Python: **`SimpleHTTPServer`**
  - Requisições GET e HEAD
  - Habilita um diretório da máquina para armazenar arquivos a serem utilizados
  - Criação (comandos executados no diretório que se deseja usar como servidor)
    - Python 2: python -m SimpleHTTPServer [porta]
    - Python 3: python -m http.server [porta]
    - Padrão: Porta 8000
  - Acesso: http://localhost:PORTA/ARQUIVO

Leitura de Dados

- Estrutura da requisição

```
d3.TIPO("arquivo")
  .then(function(data) {
    //Ações de tratamento do dado
   })
  .catch(function(error) {
    //Ações de tratamento do erro
   });
```

Exemplo

- Considere o arquivo dados.csv

```
Nome,Idade
Jose,30
Ana,32
Carla,12
Igor,67
Ed,56
Tomas,25
Julia,82
```


Exemplo

- Primeira forma

```
d3.csv("./dados.csv").then(
 function(d) {
  var table = d3.select('body').append('table');
  for (var i = 0; i < d.length; i++) {
      var tr = table.append('tr');
      tr.append('td').text(d[i].Nome);
      tr.append('td').text(d[i].Idade);
  }
 });
```

Exemplo

- Segunda forma

```
d3.csv("./dados.csv").then(
 function(d) {
  var table = d3.select('body').append('table').
      selectAll('tr');
  var tr = table.data(d).enter().append('tr');
  var td = tr.selectAll('td').data(
      function(d) {return [d.Nome,d.Idade];}).
            enter().append('td').
            text(function(d) {return d;});
 });
```


Observações

- As funções previamente apresentadas exigem que um cabeçalho esteja presente nos dados
- Além disso, todos os dados são associados como *strings*
- D3 permite formatações especiais para lidar com arquivos que tenham múltiplos dados de tipos diferentes

Observações

- Maneira usual de formatação

```
d3.csv(ARQUIVO,
 function(d) {
  return {
        NOVO_DADO1:DADO1_ANTIGO,
        NOVO_DADO2:DADO2_ANTIGO,
        ...
 };
 })
```


Observações

- Assim, é possível, dentre outras coisas:
  - Alterar nome das colunas
  - Alterar tipo de dados
  - Computar sumarizações ou estatísticas
  - Formatar como dado é armazenado

Exemplo

- Adaptado de http://learnjsdata.com/read_data.html
- Arquivo **`cities.csv`**

```
city,state,population,land area
seattle,WA,652405,83.9
new york,NY,8405837,302.6
boston,MA,645966,48.3
kansas city,MO,467007,315.0
```


Exemplo

- Adaptado de http://learnjsdata.com/read_data.html

```
d3.csv("./cities.csv",
 function(d) {
   return {
    c:d[“city”], //CITY SE CHAMARÁ C
    s:d.[“state”], //STATE SE CHAMARÁ S
    p:+d.[“population”], //POPULATION SE CHAMARÁ P, E
                         //SERÁ TRATADO COMO NÚMERO
    l:+d["land area"] //LAND AREA SE CHAMARÁ L, E
                      //SERÁ TRATADO COMO NÚMERO
   };})
```

Exemplo

- Adaptado de http://learnjsdata.com/read_data.html

```
.then(function(d) {
  var table = d3.select('body')
      .append('table').selectAll('tr');
  var tr = table.data(d).enter().append('tr');
  var td = tr.selectAll('td').data(
     function(d) {return [d.c,d.s,d.p,d.l];})
     .enter().append('td')
       .text(function(d) {return d;});});
```


Exemplo - JSON

- Considerando os seguintes dados:

```
[
  { "nome": "Ana",   "idade": 20, "nota": 8.5},
  { "nome": "Bruno", "idade": 22, "nota": 7.1},
  { "nome": "Carla", "idade": 19, "nota": 9.0},
  { "nome": "Diego", "idade": 21, "nota": 6.8}
]
```

Exemplo - JSON

- Execução:

```
d3.json("alunos.json")
  .then(function(data) {
      console.log(data);
  })
  .catch(function(error) {

console.error(error);
  });
```

```
▼ Array(4) i
  ▶ 0: {nome: 'Ana', idade: 20, nota: 8.5}
  ▶ 1: {nome: 'Bruno', idade: 22, nota: 7.1}
  ▶ 2: {nome: 'Carla', idade: 19, nota: 9}
  ▶ 3: {nome: 'Diego', idade: 21, nota: 6.8}
    length: 4
  ▶ [[Prototype]]: Array(0)
```

- OBS: dados já vem formatados nesse caso!

Exemplo - JSON

- Considerando os seguintes dados:

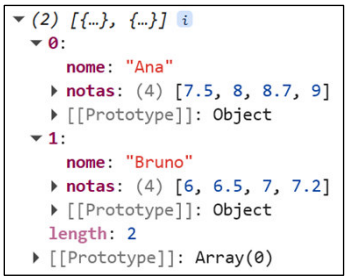
```
[{"nome": "Ana",   "notas": [7.5, 8.0, 8.7, 9.0]},
 {"nome": "Bruno","notas": [6.0, 6.5, 7.0, 7.2]}]
```

Exemplo - JSON

- Execução:

```
d3.json("alunos.json")
  .then(function(data) {
      console.log(data);
  })
  .catch(function(error) {

console.error(error);
  });
```



- Formatação difícil caso os dados estivessem em CSV!

Leitura de Dados

- **`d3.dsv(del,arquivo)`**
  - Faz a leitura de um arquivo com delimitação específica
  - **`del`**: caracter ou *string* usada para delimitação
  - **`arquivo`**: nome do arquivo a ser lido

Exemplo

Arquivo "**`/data/animals_piped.txt`**"

```
name|type|avg_weight
tiger|mammal|260
hippo|mammal|3400
komodo dragon|reptile|150
```

Fazendo a leitura do aquivo

```
d3.dsv("|", "/data/animals_piped.txt").then(function(data){
  console.log(data[1]);
});
```

Retirado de de http://learnjsdata.com/read_data.html

Leitura de Dados

- **`d3.text(arquivo)`**
  - Faz a leitura direta do arquivo, retornando uma *string* contendo todo o conteúdo desse arquivo
  - Exemplo:

```
d3.text("./cities.csv")
        .then(function (data) {
          console.log(data);
          console.log(data.length);
        });
```

Leitura de vários arquivos

- **`Promise.all([LEITURA1,LEITURA2,...])`**
  - Faz várias requisições de leitura simultaneamente
  - Cada leitura pode ser feita de maneira independente
  - Retorno: um vetor contendo os conjuntos de dados lidos, na ordem de passagem dos parâmetros

Exemplo

```
Promise.all([d3.csv("./cities.csv"),d3.csv("./dados.csv")])
.then(function(data) {
 var table = d3.select('body').append('table')
                .selectAll('tr');
 var tr = table.data(data[0]).enter().append('tr');
 var td = tr.selectAll('td').data(function(d) {
      return [d.city,d.state,d.population,d["land area"]];})
  .enter().append('td').text(function(d) {return d;});
table = d3.select('body').append('table').selectAll('tr');
tr = table.data(data[1]).enter().append('tr');
td = tr.selectAll('td').data(function(d) {
    return [d.Nome,d.Idade];})
       .enter().append('td').text(function(d) {return d;});
});
```

Combinando coleções

- **`Promisse.all`** pode ser utilizado para combinar partes de uma coleção muito grande

```
Promise.all([
    d3.csv("/data/big_data_1.csv"),
    d3.csv("/data/big_data_2.csv"),
    d3.csv("/data/big_data_3.csv")
]).then(function(allData) {
    console.log(d3.merge(allData));
});
```

Retirado de http://learnjsdata.com/combine_data.html

Formatando textos

- D3 oferece recursos para organizar a estrutura de um texto em formatos que utilizam delimitadores
- Recursos comuns
  - d3.csvParse
  - d3.tsvParse
  - d3.csvParseRows
  - d3.tsvParseRows
  - d3.csvFormat
  - d3.tsvFormat
  - d3.dsvFormat(DELIMITADOR)

(csv/tsv)Parse

- Estrutura uma *string* em objetos
- Ex: Considerando a string **`str`**:

```
"Nome,Idade\nJose,30\nAna,32\nCarla,12\nIgor,67\nEd,56"
```

- O resultado de **`d3.csvParse(str)`** será:

```
[{Nome: "Jose", Idade: "30"},{Nome: "Ana", Idade:
"32"},{Nome: "Carla", Idade: "12"},{Nome: "Igor",
     Idade: "67"},{Nome: "Ed", Idade: "56"}]
```

- Será criado um atributo *columns* com os valores: **`["Nome", "Idade"]`**
- Observação: A primeira linha deve indicar um cabeçalho!

(csv/tsv)ParseRows

- Transforma uma *string* em um vetor de tuplas de objetos
- Ex: Considerando a string str:

```
"Jose,30\nAna,32\nCarla,12\nIgor,67\nEd,56"
```

- O resultado de **`d3.csvParseRows(str)`** será:

```
[["Jose", "30"],["Ana", "32"],["Carla",
  "12"],["Igor", "67"],["Ed", "56"]]
```

- Observação: Não é necessário haver um cabeçalho!
  - Útil para manipular arquivos/dados nos quais não foi especificado um cabeçalho

(csv/tsv)ParseRows

- Exemplo de aplicação: leitura de arquivo csv sem cabeçalho

```
d3.text("./dados2.csv").then(
  function(text) {
    var f = d3.csvParseRows(text);
    d3.select('body').selectAll('p')
    .data(f).enter().append('p')
    .text(function(d) {
            return d[0] + '-' + d[1];});
});
```


(csv/tsv)Format

- Transforma uma estrutura de objetos em uma *string* estruturada
- Ex: Considerando uma estrutura **`data`**:

```
[{Nome: "Jose", Idade: "30"},{Nome: "Ana", Idade:
"32"},{Nome: "Carla", Idade: "12"},{Nome: "Igor",
     Idade: "67"},{Nome: "Ed", Idade: "56"}]
```

O resultado de **`d3.csvFormat(data)`** será:

```
"Nome,Idade\nJose,30\nAna,32\nCarla,12\nIgor,67\nEd,56"
```

(csv/tsv)FormatRows

- Transforma um vetor de tuplas de objetos em uma *string* estruturada
- Ex: Considerando o vetor de tuplas **`data`**:

```
[["Jose", "30"],["Ana", "32"],["Carla",
  "12"],["Igor", "67"],["Ed", "56"]]
```

- O resultado de **`d3.csvFormatRows(data)`** será:

```
"0,1,Jose,30\nAna,32\nCarla,12\nIgor,67\nEd,56"
```


(csv/tsv)FormatRows

- Ex: Considerando o vetor de tuplas **`data2`**:

```
[{"n":"Jose","i":"30"},{"n":"Ana","i":"32"},
{"n":"Carla","i":"12"},{"n":"Igor","i":"67"},
            {"n":"Ed","i":"56"}]
```

- O resultado de **`d3.csvFormat(data2,["n","i"])`** será:

```
"n,i\nJose,30\nAna,32\nCarla,12\nIgor,67\nEd,56"
```

Referências

- https://www.tutorialsteacher.com/d3js/loading-data-from-file-in-d3js
- https://blog.bitsrc.io/how-to-create-charts-from-external-data-sources-with-d3-js-4abbcb574706